

Design and Implementation of an Intelligent Diagnosis and Management System for Citrus Orchards

Yuanjie Shen, Qiang Lin

Beijing Information Science & Technology University, School of Management Science and Engineering

Keywords: citrus orchard, disease diagnosis, image recognition, environmental monitoring, task management

Abstract. To address experience-dependent citrus disease diagnosis, the difficulty of linking recognition results with subsequent management records, and incomplete orchard task workflows, this paper designs and implements a prototype system for intelligent citrus orchard diagnosis and management. The system adopts a browser/server architecture. Its backend is built with Rust, Axum, and PostgreSQL, while the inference layer supports both local Tract ONNX models and remote artificial intelligence services. The frontend integrates disease diagnosis, temperature and humidity data display, task management, fertilization recommendations, an administration dashboard, and a Three.js-based three-dimensional orchard view. Diagnosis is based on two-stage visual inference: non-target images are first filtered and valid images are then classified. Low-confidence results are routed for manual review, while non-healthy results that meet the threshold can create unfinished management tasks. In the current development environment, the in-memory API achieved approximately 96,900 requests per second at a concurrency level of 50, while the database API peaked at about 3,821 requests per second, with no request errors observed during the performance tests. Manual functional checks covered the main workflows, including authentication, diagnosis interfaces, environmental sampling, task management, and administration. These checks demonstrate interface and workflow behavior only; they do not establish diagnostic accuracy or field usability. The system provides a deployable prototype for the digital management of small and medium-sized citrus orchards.

柑橘果园智能诊断与管理系统设计与实现

沈瑗杰 林强

北京信息科技大学 管理科学与工程学院

关键词: 柑橘果园; 病害诊断; 图像识别; 环境监测; 任务管理

中文摘要. 针对柑橘病害诊断依赖人工经验、识别结果难以形成后续管理记录、农事任务缺少闭环等问题, 本文设计并实现了一套柑橘果园智能诊断与管理原型系统。系统采用 B/S 架构, 后端基于 Rust、Axum 和 PostgreSQL 构建, 推理层支持 Tract ONNX 本地模型与远端人工智能服务两种运行方式; 前端集成病害识别、温湿度数据展示、任务管理、施肥建议、管理员后台及 Three.js 三维果园沙盘。病害诊断以两阶段视觉推理为基础, 先过滤非目标图像, 再完成病害分类; 低置信度结果进入人工复核, 达到阈值的非健康结果可自动创建未完成治理任务。性能测试结果表明, 在本次开发环境下, 纯内存接口在并发数为 50 时达到约 9.69 万次/s, 数据库接口峰值约为 3 821 次/s, 性能测试过程未出现请求错误。人工功能验证覆盖认证、诊断接口、环境采样、任务管理和后台管理等主要流程。上述测试仅反映接口与功能流程, 不作为模型诊断准确率或田间可用性的证据。该系统为中小型柑橘果园数字化管理提供了可部署的原型实现。

1. 引言

柑橘生产容易受到黄龙病、溃疡病、黑斑病、沙皮病等病害影响。传统诊断主要依赖人工巡查和专家经验, 不仅需要较高的人力成本, 而且容易受到巡检范围、人员疲劳和早期症状不明显等因素限制。近年来, 卷积神经网络和目标检测算法逐渐应用于作物叶片病害识别, 研究重点也由受控背景下的图像分类, 发展到自然场景下的小目标检测、严重程度分级和轻量化部署^[1-3,5-8]。郑争兵等针对自然果园中背景复杂、光照变化和病斑目标较小等问题改进 YOLOv5,

提高了柑橘叶片病害小目标的检测能力^[3]；Zhu 等将柑橘叶片病害分类与严重程度分级结合，实现了病害类别识别和等级判定^[6]；2026 年的相关研究进一步关注计算效率和果园图像的实际应用能力^[5]。

尽管深度学习模型在公开数据集上取得了较高精度，农业视觉仍面临样本标注成本高、跨场景泛化不足、病斑相似和复杂环境干扰等问题^[1-2]。不同研究使用的数据集、拍摄环境和评价指标并不完全一致，因此实验室或公开数据集上的高准确率不能直接等同于真实果园中的稳定识别效果^[5-8]。对于实际管理系统，仅给出一个病害类别还不够，还需要结合历史记录和治理任务形成可执行的管理流程。

果园管理还需要把环境采样、诊断记录和任务执行信息放到同一服务中。与此同时，数字孪生和三维可视化可以用于果园空间状态展示与交互，相关研究也展示了其在树级管理中的应用潜力^[4,9-10]。本文将三维模块定位为管理界面，而不是具有完整同步和仿真能力的数字孪生系统。

现有研究多聚焦于病害识别或三维展示中的单一环节，完整覆盖诊断记录、任务生成和执行追踪的应用仍相对不足^[1-3,4,9-10]。针对这一问题，本文设计并实现一套柑橘果园智能诊断与管理系统，主要工作如下：

- 构建以两阶段视觉推理为核心、支持本地 ONNX 与远端人工智能服务切换的病害诊断流程，实现非目标图像过滤和病害分类；
- 将环境采样、诊断记录、任务管理、施肥建议和三维果园展示整合到统一系统中，支持根据有效诊断结果或环境风险创建任务；
- 实现用户注册、身份认证、权限控制、注册审核、动态设置和审计日志等管理功能，保证管理操作可控制、可追溯；
- 对系统接口性能及主要功能流程进行测试，并明确当前原型在独立诊断评测和田间试用方面的局限。

1.1 研究场景、系统边界与贡献定位

本文面向具备智能手机或计算机终端、能够由种植人员上传叶片图像并维护基本记录的中小型柑橘果园。系统将图像、用户手动提交的环境读数 and 任务记录统一保存；当前版本不直接连接田间传感器，也不替代植保人员的现场诊断。图片不符合目标叶片条件、诊断置信度低于阈值或用户认为结果不合理时，均应重新采集或交由人工复核。

本文的贡献定位为诊断结果与管理记录之间的流程整合和可部署实现，而非提出新的病害识别网络、证明诊断精度优于已有模型，或证明已经替代人工巡检。与相关工作在本文可核验范围内的比较如表 1 所示。

表 1 相关工作与本文系统的功能定位比较

工作类型	主要关注点	是否覆盖管理闭环	与本文的关系
柑橘叶片识别研究 ^[3,5-8]	病害检测、分类或分级	通常不涉及	为视觉诊断模块提供方法背景，不作为管理平台对比结论
果园三维/数字孪生研究 ^[4,9-10]	空间展示、交互或数字孪生	侧重场景表达	本文仅实现状态可视化，不宣称完整数字孪生
本文系统	诊断记录、环境记录与治理任务关联	支持基本关联与任务追踪	工程原型；诊断效果仍需独立数据验证

2. 系统设计

2.1 用例设计

系统用户分为普通用户和管理员两类。

普通用户需要完成注册、登录和退出操作，上传柑橘叶片图像进行病害识别，查看预测类别、置信度、严重程度及防治建议；同时能够浏览温湿度采样记录、健康点统计和农事任务，并通过三维沙盘查看果园布局和果树状态。用户还可以根据历史诊断记录或手动输入的土壤和果树参数生成施肥建议。

管理员除具备普通用户功能外，还需要管理用户和邀请码、审核待注册用户、查看审计日志和统计信息，并按用户维度查询果园数据。部分系统设置支持运行期间动态调整，减少因修改参数而重新部署服务的需要。

在非功能需求方面，系统需要实现数据持久化、访问权限控制、异常处理和操作追溯；推理模块应同时支持离线与联网环境；前端静态资源、HTTP API 和识别图片归档应能够通过同一服务端口访问，从而降低部署复杂度。

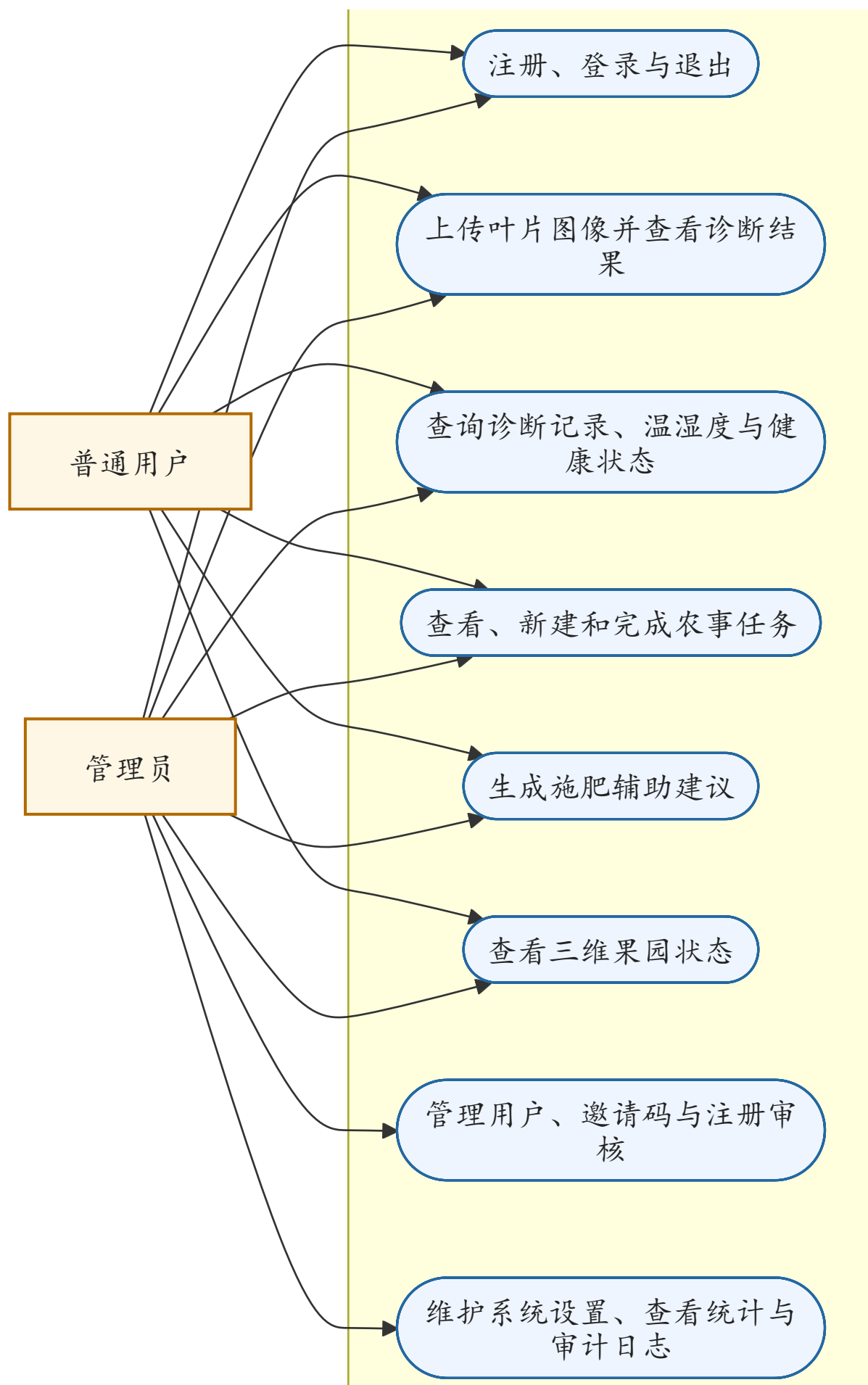


图 1 系统用例图

2.2 两阶段视觉识别与人工复核流程

深度学习已成为叶片病害分类和检测的主要技术路线，但实际图像可能包含非叶片、非柑橘或质量较差的输入^[2,5-8]。为降低无关图像直接进入病害分类模型所造成的误判，系统采用两阶段视觉推理：

- 1. 目标门控阶段：对上传图片进行格式校验、尺寸调整和归一化处理，由第一阶段模型判断图像是否属于可识别的目标叶片；
- 2. 病害分类阶段：通过门控的图像进入第二阶段模型，输出健康状态或具体病害类别及其概率分布。

人工复核分流：对病害分类结果应用置信度阈值；低于阈值的结果标记为待人工复核，不自动生成治理任务。

两阶段视觉识别构成默认链路。代码中曾实现一项仅在显式传入温湿度时启用的规则化辅助重加权，但默认识别页面没有接入该数据链路，且其参数缺少独立数据验证。为避免将未经验证的规则作为研究结论，本文不把该功能列为方法贡献或实验结果，仅将其保留为后续研究方向。

系统提供两种推理模式。在本地模式下，Tract 加载 ONNX 模型完成视觉推理，适用于网络受限和数据不宜外传的环境；在远端模式下，本地模型先进行目标门控，再将有效图像交由 OpenRouter 生成病害分析结果。两种模式的结果均需与管理员设定的置信度阈值比较，低于阈值时标记为待人工复核。

2.3 模型实现、参数与可追溯性

两阶段模型均由项目早期的 PyTorch 实现导出为 ONNX，再由 Tract 在服务端加载。第一阶段为二分类模型，类别为“非果树”和“是果树”；第二阶段为四分类模型，类别依次为黄龙病、健康果树、溃疡病和沙皮病。二者均采用 EfficientNet-B0 作为特征提取骨干，并在其后接两层全连接分类头（128 个中间单元、Dropout 和 ReLU）。输入图像统一转换为 RGB、缩放至 224 x 224，并按 ImageNet 均值 (0.485, 0.456, 0.406) 与标准差 (0.229, 0.224, 0.225) 标准化。

表 2 两阶段 ONNX 模型规格

阶段	任务	类别	模型与输入	输出与后续处理
第一阶段	目标门控	非果树；是果树	EfficientNet-B0； RGB 224 x 224	二类 logits；非目标图像终止后续分类
第二阶段	病害分类	黄龙病；健康果树；溃疡病；沙皮病	EfficientNet-B0； RGB 224 x 224	四类 logits；Softmax 后进入阈值分流

原始训练程序显示使用随机裁剪、水平翻转和颜色扰动进行训练，批量大小为 16、训练轮数为 20、优化器为 AdamW，学习率为 1e-4，权重衰减为 0.01，并采用余弦退火调度。训练和验证图像的原始目录、采集来源、授权信息及当时的数据划分未被完整保留在当前项目中。因此，本文仅将这两个模型作为原型系统的集成组件，报告其结构、类别和推理预处理；不以权重文件名中的历史数字作为诊断性能证据，也不对模型许可或独立泛化能力作未经核验的声明。

系统设置中置信度阈值的默认值为 0.75，允许管理员在 0.30 至 0.99 的范围内调整。该值是为了降低自动创建任务风险的工程初始配置，并非经独立验证集选出的最优阈值。当前规则为：非果树不创建任务；健康结果不创建任务；低于阈值的结果进入人工复核；仅非健康且不低于阈值的结果自动创建治理任务。阈值的准确率、误报率和人工复核比例需在独立标注集上进一步评估。

2.4 管理闭环

诊断完成后，系统将结果写入诊断记录表；对于通过目标门控、判定为非健康且置信度达到管理员设定阈值的结果，系统可以自动创建一条未完成的治理任务。低于阈值的结果仅标记为待人工复核，不自动创建治理任务；环境风险接口也可以创建环境监测任务。任务保存风险等级、类型、来源、创建时间和完成时间，并支持查询、新建和完成操作。施肥建议模块提供两类 OpenRouter 生成接口：一类根据最近诊断记录生成摘要，另一类根据土壤、pH、养分水平、生长阶段、树龄和面积生成结构化方案；结果定位为辅助建议，实际用量仍需结合土壤检测和当地农艺规范确定。

通过上述流程，系统将图像识别结果转换为可查询的诊断记录和可完成的治理任务，形成从识别到任务记录的基本关联。

3. 系统实现

系统采用三层 B/S 架构，由表现层、服务层和数据层组成。相关研究表明，三维场景能够增强果园空间状态展示和人机交互能力^[4,9-10]。本文系统借鉴这一思路，但当前 Three.js 模块主要承担数据可视化功能，尚不构成具有完整双向同步和仿真能力的数字孪生系统。

系统总体架构如图 1 所示，各层的主要功能如表 1 所示。

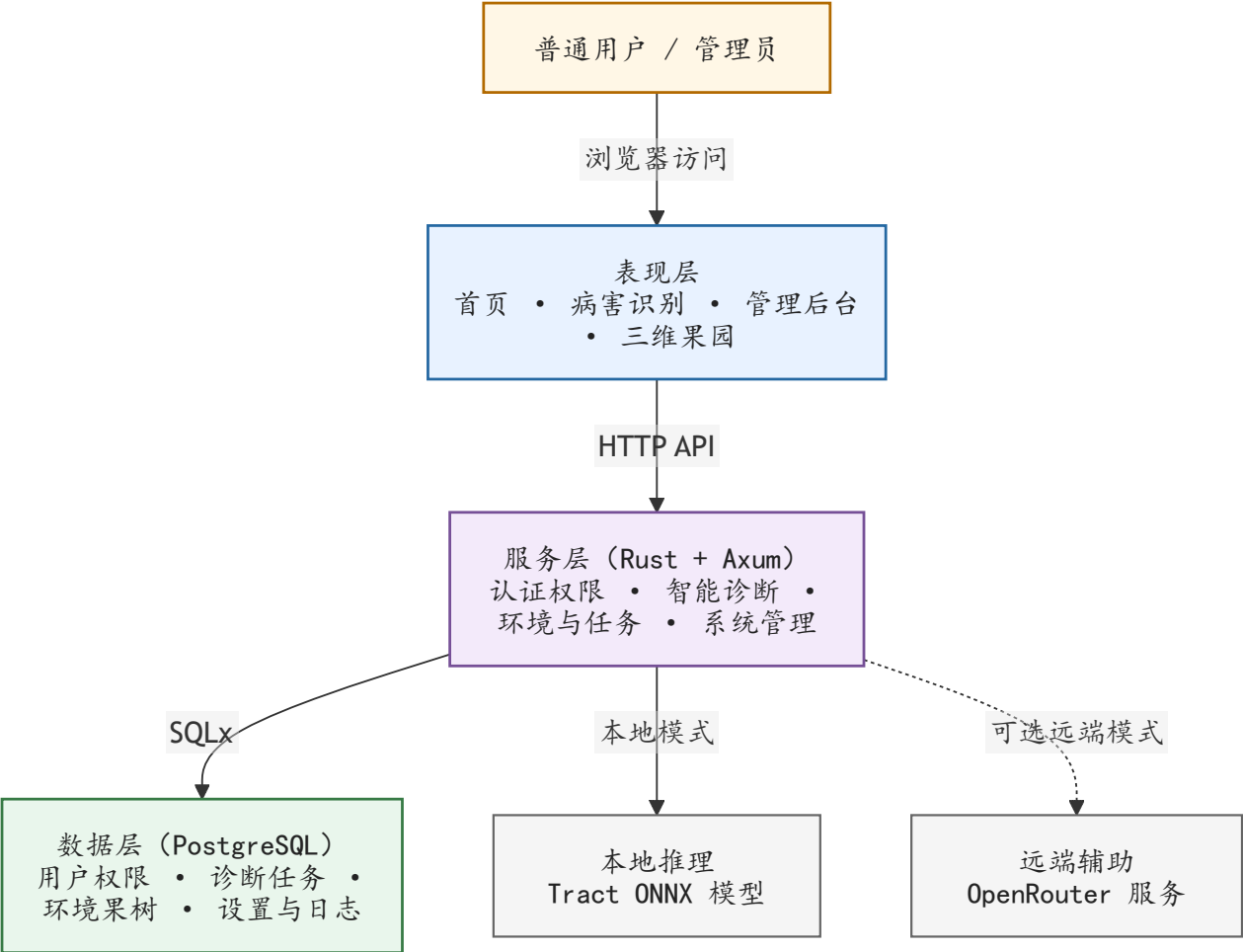


图 2 系统总体架构图

表 3 系统分层功能表

表现层	服务层	数据层
公开首页 病害识别页 管理后台 三维果园沙盘	用户与会话 智能诊断 环境采样与任务管理 施肥建议 系统设置与审计	用户与权限数据 诊断与任务数据 环境与果树数据 系统设置与审计日志

表现层由原生 HTML、CSS 和 JavaScript 页面组成，主要包括公开首页、病害识别页、管理员页面和三维果园页面。识别页负责图片上传、结果展示和历史记录查询；管理员页面展示用户、审核、系统设置和审计信息；三维页面使用 Three.js 绘制地形、果树和状态标记。

三维沙盘根据数据库中的果树坐标、地形高度和最近诊断结果生成场景，并通过颜色、图标和信息面板表达不同状态。与相关果园数字孪生研究相比^[4,9]，当前实现重点是低成本的浏览器端可视化，没有建立复杂生长模型和实时仿真模型，适合用作果园状态总览界面。

服务层基于 Rust 和 Axum 构建，由 server.rs 统一注册路由，并通过共享状态管理数据库连接池、推理运行时和远端人工智能客户端。主要模块如下：

- `handlers_core/`: 处理环境采样、任务、统计和历史记录接口；
- `handlers_ai/`: 处理图片上传、病害诊断和施肥建议；
- `user_routes/`: 实现注册、登录、会话验证、用户审核及管理员接口；
- `inference/`: 封装 ONNX 本地推理、远端辅助分析和运行模式切换；
- `client/`: 封装远端 HTTP 请求与结构化结果解析；
- `bootstrap.rs`: 在首次启动时完成数据表创建和演示数据初始化。

采用统一封装的主要业务 API 请求与响应流程如图 2 所示。

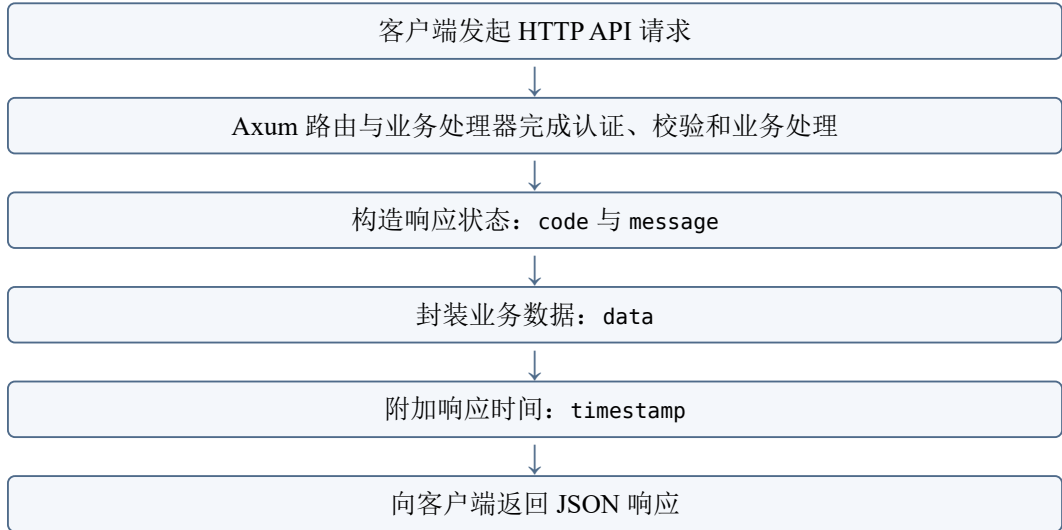


图 3 主要业务 API 的统一请求与响应流程

图 2 所示的统一响应结构适用于当前主要业务 API，不代表系统内所有接口均采用相同格式。页面和管理员接口需要携带有效 Token；管理员接口在身份验证后继续校验角色，避免普通用户访问用户管理、系统设置和审计数据。部分历史 API 仍使用参数查询或独立的响应结构。

系统使用 PostgreSQL 存储业务数据，通过 SQLx 连接池进行异步访问。数据库包含 11 张核心表，可分为四类，具体如表 2 所示。

表 4 系统核心数据表

数据类别	主要数据表	作用
用户权限	<code>app_users</code> 、 <code>app_sessions</code> 、 <code>app_invitations</code> 、 <code>app_pending_users</code>	保存用户、会话、邀请码和待审核注册信息
业务记录	<code>app_diagnosis_records</code> 、 <code>app_tasks</code>	保存诊断结果和农事任务
环境监测	<code>app_temperature_humidity</code> 、 <code>app_tree_sensor_records</code> 、 <code>app_orchard_trees</code>	保存环境采样、树级传感器记录和果树空间信息
系统管理	<code>app_system_settings</code> 、 <code>app_admin_audit_logs</code>	保存动态配置和管理员操作日志

诊断记录保存最终分类结果、诊断时提供的环境数据及相关建议，支持历史查询。任务表保存来源、风险等级、任务类型、完成状态和时间信息，使病害诊断能够与治理任务关联；当前尚未保存原始概率、调整后概率、负责人或状态变更历史。

系统采用单端口部署方式，由 Axum 同时提供静态文件、HTTP API 和识别图片访问。主要运行条件包括 Rust 1.85 及以上版本、PostgreSQL、ONNX 模型文件和可选的远端人工智能服务密钥。

系统配置与启动流程如图 3 所示。

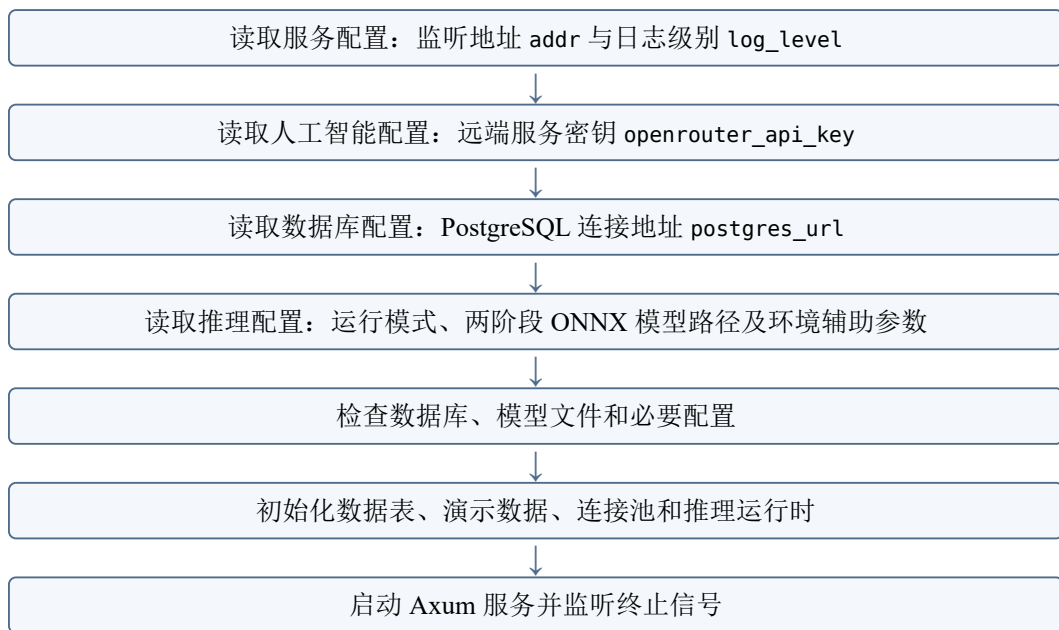


图 4 系统配置与启动流程

启动流程为：准备数据库和模型文件，完成配置后执行 `cargo run --release`。系统启动时检查数据表并初始化演示数据，随后通过指定端口访问前端页面。服务监听终止信号并执行优雅关闭。

4. 系统测试

系统测试包括接口性能测试和功能集成测试。性能测试在 **Release** 模式下运行，使用自定义 PowerShell 脚本分别测试健康检查、静态首页、纯内存接口和数据库接口。每组持续 8 s，并设置 1、10 和 50 三种并发级别。测试不包含 ONNX 模型推理和远端人工智能请求，因此结果仅反映 Web 服务、静态资源和数据库查询部分的性能，不能证明诊断模型的准确性或端到端诊断时延。

由于当前记录未包含处理器型号、内存、操作系统负载和网络拓扑等完整硬件信息，以下数据只能用于分析本次部署环境中的相对性能，不能直接与其他研究或不同设备的测试结果进行横向比较。Web 服务性能测试结果如表 3 所示。

表 5 Web 服务性能测试结果

测试项	并发	请求数	RPS	平均/ms	P50	P95	P99	错误率
health	1	83 055	10 381.74	0.095	0.089	0.122	0.145	0%
health	10	510 815	63 850.96	0.156	0.140	0.235	0.344	0%
health	50	761 620	95 196.48	0.524	0.472	0.872	1.352	0%
static index	1	26 704	3 337.93	0.299	0.288	0.355	0.467	0%
static index	10	106 944	13 367.27	0.747	0.728	0.936	1.146	0%
static index	50	97 261	12 154.26	4.112	4.033	5.171	5.824	0%
memory api	1	80 491	10 061.27	0.099	0.094	0.115	0.152	0%
memory api	10	546 148	68 067.54	0.146	0.135	0.210	0.288	0%
memory api	50	774 995	96 870.76	0.515	0.453	0.899	1.485	0%
database api	1	14 746	1 843.18	0.542	0.532	0.620	0.703	0%
database api	10	30 576	3 820.91	2.616	2.584	3.034	3.318	0%
database api	50	30 466	3 802.73	13.138	13.002	14.570	16.365	0%

健康检查和纯内存接口的吞吐量随并发数增加而提升，在并发数为 50 时分别达到约 9.52 万次/s 和 9.69 万次/s，P95 均低于 0.9 ms。静态首页在并发数为 10 时达到最高吞吐量，继续增加并发后吞吐量下降且延迟上升，说明静态文件传输相较于短 JSON 响应具有更高的数据复制和传输开销。

数据库接口在并发数为 10 时达到约 3 821 次/s，并发增加至 50 后吞吐量基本不再增长，平均延迟由 2.616 ms 上升至 13.138 ms。该现象与数据库连接池上限为 3 有关：请求数量超过可用连接后，需要在连接池中排队等待。全部测试均未出现错误，表明系统在短时压力下保持了稳定响应，但数据库连接池大小仍应根据实际部署资源和业务负载进行调整。

人工功能验证覆盖认证、病害诊断接口、环境采样、任务管理、管理员功能和数据持久化等流程，主要功能测试内容如表 4 所示。

表 6 主要功能测试内容

测试类别	主要测试项	预期结果
用户认证	开放注册、邀请码注册、重复用户名、登录、登出	合法操作成功；重复或错误凭证被拒绝；登出后 Token 失效
权限控制	未登录访问保护页面、普通用户访问管理员接口	返回认证失败或权限不足，前端执行相应跳转
病害诊断	上传有效叶片、非目标图片、查询历史记录	返回完整诊断字段；非目标图片被门控；可查询最近识别记录
环境监测	提交温湿度数据，查询最新值与历史值	写入与读取结果一致，数据能够被相关接口查询
任务管理	新建、查询、完成任务，根据病害或环境自动生成任务	未完成和已完成状态正确持久化，自动任务包含来源信息
管理后台	用户管理、邀请码、审核、系统设置、审计日志	管理员可操作；普通用户无权访问；操作写入审计记录
数据初始化	首次启动建表、默认设置和演示果树初始化	11 张核心表创建成功，空表时写入演示树位和传感器数据

接口验证使用正常参数、缺失参数、无效 Token、越权访问和不存在资源等场景，检查 HTTP 状态和返回字段。不同接口的响应结构并不完全一致，不能将所有接口概括为同一 JSON 格式。数据验证确认识别记录、任务、图片路径和环境数据能够正确写入数据库。安全检查确认密码字段以 BLAKE3 摘要保存而非明文，真实配置文件和密钥不进入版本控制。

4.1 诊断评测与实地验证的限制

功能测试中“上传有效叶片返回完整字段”仅验证了接口链路、结果持久化和任务分流规则，不能替代诊断有效性评测。项目备份中虽保留了按类别存放的图像文件，但没有可追溯的训练/测试划分、采集与标注记录；将其直接作为测试集会生成训练数据重叠风险。因此，本次修改不报告门控准确率、四分类 Top-1 准确率、分类别 Precision/Recall/F1 或环境辅助效果，亦不将 Web 接口吞吐量作为模型性能证据。

后续验证应构建与训练数据独立的标注集，分别报告门控二分类指标、四分类的 Top-1 准确率和各类别 Precision、Recall、F1，以及混淆矩阵；并应以多组阈值统计自动建任务比例、人工复核比例及相应误报/漏报情况。系统当前仅在开发环境完成了功能和短时性能测试，尚未开展可报告的真实果园试用；因此本文将其定位为可部署的原型系统，而非已经过田间验证的应用系统。

5. 结论

本文设计并实现了一套集病害诊断、环境监测、任务管理、施肥建议、后台管理和三维可视化于一体的柑橘果园智能管理原型系统。系统以两阶段视觉推理完成非目标图像过滤和细粒度分类；达到置信度阈值的非健康诊断结果能够生成未完成治理任务，低置信度结果则进入待人工复核状态，从而形成从识别到任务记录的基本关联。

系统后端基于 Rust、Axum 和 PostgreSQL 实现，本地推理采用 Tract ONNX，同时保留远端人工智能辅助分析能力。性能测试表明，在本次环境下，纯内存接口具有较低延迟，数据库接口性能主要受到连接池规模限制；功能测试验证了用户认证、病害诊断、环境采样、任务管理和管理员功能的基本可用性。

本文没有以当前功能测试证明模型诊断准确率，亦没有将实验室/开发环境结果外推为田间效果。后续工作将重点完善病害模型的独立测试和田间验证，补充 HTTP 集成测试，接入真实传感器并完善环境数据管理，增加任务处理中状态、负责人分派和状态历史。同时，可进一步优化三维场景的大规模渲染和实时数据同步，使系统逐步由状态可视化平台向数据驱动的果园管理与决策平台扩展。

References

[1] XIAO D Q, LIU Q, PAN Q Y, et al. Low-annotation learning in agricultural vision: A review of semi-supervised, weakly supervised, and self-supervised methods. Journal of South China Agricultural University, 2026, 47(3): 369-381. doi: 10.7671/j.issn.1001-411X.202601035.

- [2] ZHU R, ZHANG J Y, HUANG J C, et al. Research progress of crop leaf disease detection based on convolutional neural networks. *Transactions of the Chinese Society of Agricultural Engineering*, 2025, 41(17): 15-28. doi: 10.11975/j.issn.1002-6819.202502117.
- [3] ZHENG Z B, ZHANG Y B, SUN L C, et al. Small-target detection method for citrus leaf diseases based on improved YOLOv5. *Transactions of the Chinese Society of Agricultural Engineering*, 2025, 41(21): 203-211. doi: 10.11975/j.issn.1002-6819.202505148.
- [4] WANG H J, LIN J Q, ZOU X J, et al. Construction of an orchard virtual interaction system based on digital twins. *Journal of System Simulation*, 2024, 36(6): 1493-1508. doi: 10.16182/j.issn1004731x.joss.23-0317.
- [5] GOYAL P, GILL J, GOYAL R, et al. Deep learning-based citrus plant disease classification using a computationally efficient CNN model. *Scientific Reports*, 2026, 16: 19316. doi: 10.1038/s41598-026-50684-y.
- [6] ZHU H, WANG D, WEI Y, et al. YOLOv8-CMS: a high-accuracy deep learning model for automated citrus leaf disease classification and grading. *Plant Methods*, 2025, 21: 88. doi: 10.1186/s13007-025-01396-3.
- [7] BUTT N, IQBAL M M, RAMZAN S, et al. Citrus diseases detection using innovative deep learning approach and hybrid meta-heuristic. *PLOS ONE*, 2025, 20(1): e0316081. doi: 10.1371/journal.pone.0316081.
- [8] GOYAL A, LAKHWANI K. Integrating advanced deep learning techniques for enhanced detection and classification of citrus leaf and fruit diseases. *Scientific Reports*, 2025, 15: 12659. doi: 10.1038/s41598-025-97159-0.
- [9] KIM S, HEO S. An agricultural digital twin for mandarins demonstrates the potential for individualized agriculture. *Nature Communications*, 2024, 15: 1561. doi: 10.1038/s41467-024-45725-x.
- [10] TAGARAKIS A C, BENOS L, KYRIAKARAKOS G, et al. Digital twins in agriculture and forestry: a review. *Sensors*, 2024, 24(10): 3117. doi: 10.3390/s24103117.